Big Data Analytics

PROJECT
REPORT

NYC Collision Data Analytics Pipeline

Prepared for: Ms. Zahida Kausar

Presented by:

Huda Noor (458675)

Humna Tariq (460497)

Table of Contents

1. Introduction 4

2. Dataset Overview 4

3. Milestone 1 - Data Ingestion 4

Tasks Performed 4

Validation 5

4. Milestone 2 - ETL Pipeline 5

4.1 ETL Transformations 5

4.2 Star Schema Design 6

4.3 A2 Traceability Table 6

4.4 Row Count Verification 7

- Only 1 duplicate record removed 8
- No unexpected data loss 8
- Dimension tables are smaller due to aggregation and uniqueness constraints 8

4.3 Validation Check 8

5. Milestone 3 – Analytics 8

5.1 Business Questions, SQL Approach, and Interpretation 8

5.2 Query Results and Insights 9

Q1 Output (sample) 9

Q2 Output (sample) 10

Q3 Output (sample) 10

Q4 Output (sample) 11

Q5 Output (sample) 11

6. Visualizations 12

6.1 Chart 1 — Fatality Trends 12

6.2 Chart 2 — Pedestrian Injury Heatmap 12

6.3 Chart 3 — Crash Severity by Borough 13

6.4 Chart 4 — Cyclist Trends 14

6.5 Chart 5 — Factors involved in multi-vehicle crashes 14

6.6 Chart 6 — Geographic Crash Density 15

7. HDFS Partition Verification 16

9. Challenges 17
10. Conclusion 17
11. GitHub Repository Link 17

1. Introduction

This project implements a complete big data pipeline using the NYPD Motor Vehicle Collisions dataset. The system begins with raw data ingestion into Hadoop HDFS, transforms the dataset into a structured warehouse using PySpark, and then performs Spark SQL analytics to answer business questions. The final output includes cleaned warehouse tables, optimized Parquet storage, analytical query results, visualizations, and business insights.

The purpose of the project is to demonstrate the complete warehouse lifecycle: ingestion, ETL transformation, dimensional modelling, loading, validation, analytical querying, visualization, and performance optimization.

2. Dataset Overview

Dataset: NYPD Motor Vehicle Collisions dataset.

Attribute	Description
Dataset Name	NYPD Motor Vehicle Collisions - Crashes
Source	NYC Open Data
Format	CSV
Approximate Size	About 2.25 million records and 29 columns
Domain	Traffic crashes, injuries, fatalities, locations, vehicle types, and contributing factors

The dataset contains crash-level records for motor vehicle collisions. Each record includes date, time, borough, coordinates, injury counts, fatality counts, vehicle types, and contributing factors. These fields support safety analysis, location-based analysis, time-series analysis, and severity analysis.

3. Milestone 1 - Data Ingestion

Milestone 1 focused on preparing the environment, validating the raw dataset, and uploading the approved dataset to Hadoop HDFS.

Tasks Performed

- Verified the CSV format and structure.
- Configured Python, Java, Hadoop, HDFS, and YARN on Windows.

- Uploaded the raw NYPD collision CSV dataset to HDFS.
- Verified the HDFS raw storage path.
- Generated ingestion proof screenshots and logs.

Validation

The HDFS raw directory was checked after upload. The presence of the dataset in HDFS confirmed that the ingestion stage was completed successfully.

```
D:\sem 6\BDA\project\prj>C:\hadoop-3.3.6\bin\hdfs.cmd dfs -ls /warehouse/processed/nypd_collisions/
Found 5 items
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:38 /warehouse/processed/nypd_collisions/dim_date
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:39 /warehouse/processed/nypd_collisions/dim_factor
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:38 /warehouse/processed/nypd_collisions/dim_location
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:38 /warehouse/processed/nypd_collisions/dim_vehicle
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:41 /warehouse/processed/nypd_collisions/fact_crash
```

4. Milestone 2 - ETL Pipeline

Milestone 2 transformed the raw crash dataset into a structured warehouse. The ETL process was implemented in PySpark and produced a star schema consisting of one fact table and four dimension tables.

4.1 ETL Transformations

A2 / Profiling Issue	Transformation Applied	PySpark Function / Method
COLLISION_ID contained null or blank values	Removed invalid crash identifiers	filter(), isNotNull(), trim()
Duplicate crash records existed	Removed duplicate records using COLLISION_ID	dropDuplicates()
Latitude and longitude were stored as strings	Converted coordinates to numeric type	cast("double")
Zero coordinates represented invalid locations	Replaced zero coordinates with null	when(), otherwise()
Some coordinates were outside plausible NYC bounds	Nullified out-of-range latitude/longitude values	filter(), when()
Crash date was stored as string	Parsed date into Spark DateType	to_date()
Crash time was separate from crash date	Built timestamp support columns	concat_ws(), to_timestamp()

Metric columns were strings	Converted injury/fatality metrics to integers	cast("int")
Many crashes had no injuries or deaths	Created property_damage_only flag	withColumn(), boolean expressions
Extreme injury records existed	Created injury_count_flagged outlier indicator	when()
Nulls caused dimension join mismatch	Created normalized join columns	coalesce(), trim(), lit()

4.2 Star Schema Design

A star schema was selected because it supports simpler analytical queries and improves readability for business analysis. The central fact table stores measurable crash outcomes and links to descriptive dimension tables.

Table	Purpose	Main Columns
fact_crash	Stores crash metrics and foreign keys	collision_id, date_key, location_key, vehicle_key, factor_key, injury/fatality measures, flags
dim_date	Stores temporal attributes	date_key, crash_date, crash_time, hour, day_of_week, month, year, is_weekend
dim_location	Stores crash location details	location_key, borough, zip_code, street, latitude, longitude
dim_vehicle	Stores vehicle type combinations	vehicle_key, vehicle_type_1 to vehicle_type_5
dim_factor	Stores contributing factor combinations	factor_key, factor_1 to factor_5

4.3 A2 Traceability Table

A2 Data Quality Issue	Transformation Applied	PySpark Function Used
Null/blank COLLISION_ID	Removed invalid records	filter(), trim()

Duplicate records	Deduplicated using ID	dropDuplicates()
Zero coordinates	Converted to null	when()
Out-of-range coordinates	Set to null	when()
Invalid date format	Parsed valid dates	to_date()
Missing crash time	Filled default value	coalesce()
Injury columns as string	Converted to integer	cast()
Missing categorical values	Replaced with defaults	coalesce(), lit()
Join mismatches	Normalized join columns	trim(), coalesce()

4.4 Row Count Verification

Raw rows: 2,256,490 → Final fact rows: 2,256,489

Stage	Row Count
Raw Data	2,256,490
After ID Cleaning	2,256,489
After Deduplication	2,256,489
After Date Cleaning	2,256,489
Fact Table	2,256,489
dim_date	1,282,181
dim_location	544,241
dim_vehicle	17,488
dim_factor	5,960

- Only 1 duplicate record removed
- No unexpected data loss
- Dimension tables are smaller due to aggregation and uniqueness constraints

4.3 Validation Check

ETL success:

```
FINAL ETL SUMMARY
====================================================================
Raw rows                                : 2256490
Rows after dedup                        : 2256488
Rows after date cleaning                : 2256488
dim_date rows                           : 1282180
dim_location rows                       : 484295
dim_vehicle rows                        : 17488
dim_factor rows                         : 5960
fact_crash rows                         : 2256488
Status                                  : SUCCESS
```

Processed HDFS:

```
D:\sem 6\BDA\project\prj>C:\hadoop-3.3.6\bin\hdfs.cmd dfs -ls /warehouse/processed/nypd_collisions/
Found 5 items
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:38 /warehouse/processed/nypd_collisions/dim_date
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:39 /warehouse/processed/nypd_collisions/dim_factor
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:38 /warehouse/processed/nypd_collisions/dim_location
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:38 /warehouse/processed/nypd_collisions/dim_vehicle
drwxr-xr-x   - DELL supergroup          0 2026-04-23 03:41 /warehouse/processed/nypd_collisions/fact_crash
```

5. Milestone 3 – Analytics

Milestone 3 built the analytics layer on top of the warehouse. The optimized Parquet warehouse was loaded into Spark, dimension tables were joined with the fact table, and Spark SQL queries were executed to answer the five business questions.

5.1 Business Questions, SQL Approach, and Interpretation

Question	SQL / Analytical Approach	Expected Output
Q1: Borough fatality rates year-over-year	GROUP BY borough and year; calculate fatalities per 1000 crashes; use RANK() window function	Trend table and line chart by borough-year
Q2: Peak hours for pedestrian injuries and contributing factors	GROUP BY hour, day_of_week, factor_1; SUM pedestrian injuries	Heatmap by day and hour plus ranked factor table

Q3: Vehicle type vs fatal/non-fatal crash breakdown by borough	CASE WHEN persons_killed > 0; GROUP BY borough, vehicle_type, severity	Stacked bar chart
Q4: Cyclist injury trends monthly since 2012	GROUP BY year and month; SUM cyclist injuries; use LAG() window function for month-over-month change	Monthly time-series line chart and pre/post-2019 summary
Q5: Contributing factors in multi-vehicle crashes with 3+ injuries	WHERE persons_injured >= 3 and vehicle_type_2 IS NOT NULL; use ROW_NUMBER() by time of day	Top-N ranked table and bar chart

5.2 Query Results and Insights

Q1 Output (sample)

```
Q1 - BOROUGH FATALITY RATES YEAR-OVER-YEAR
==========================================================================================
 borough  year  total_crashes  total_fatalities fatality_rate_per_1000_crashes  fatality_rank_within_year
   BRONX  2012           9511                17                1.78740405845 86                          2
   BRONX  2013          19768                32                1.6187778227438                           2
   BRONX  2014          19688                19                0.9650548557497                           4
   BRONX  2015          21258                26                1.2230689622730                           3
   BRONX  2016          21660                19                0.8771929824561                           3
   BRONX  2017          21397                17                0.7945039024162                           5
```

Interpretation

This query calculates fatality rates per 1000 crashes for each borough and year. The result helps compare crash severity fairly because it normalizes fatalities by crash volume.

A borough with fewer crashes can still have a high fatality rate, so this analysis is more useful than total crash count alone. The RANK() window function ranks boroughs by fatality rate within each year.

Business action: Transportation planners can prioritize borough-year combinations with high fatality rates for enforcement, road design review, and targeted safety campaigns.

Q2 Output (sample)

```
Q2 - PEAK HOURS FOR PEDESTRIAN INJURIES + CONTRIBUTING FACTORS
==========================================================================================
 hour  day_of_week contributing_factor  total_pedestrian_injuries
   17           6         Unspecified                        671
   17           5         Unspecified                        652
   18           3         Unspecified                        643
   18           5         Unspecified                        633
   17           3         Unspecified                        622
   18           6         Unspecified                        612
```

Interpretation

This query identifies combinations of hour, day of week, and contributing factor associated with pedestrian injuries. It supports understanding when pedestrians face the highest risk.

The heatmap makes it easier to detect high-risk time windows. If injuries cluster during commuting or evening hours, agencies can adjust signal timing, enforcement schedules, or pedestrian education efforts.

Business action: city safety teams can focus resources during peak injury periods instead of applying the same intervention uniformly across all times.

Q3 Output (sample)

```
Q3 - VEHICLE TYPE VS FATAL/NON-FATAL CRASH BREAKDOWN BY BOROUGH
==========================================================================================
  borough                      vehicle_type_1 crash_severity  crash_count
 BROOKLYN                                Sedan      Non-Fatal       147622
   QUEENS                                Sedan      Non-Fatal       122339
 BROOKLYN Station Wagon/Sport Utility Vehicle      Non-Fatal       113651
   QUEENS Station Wagon/Sport Utility Vehicle      Non-Fatal       103590
 BROOKLYN                    PASSENGER VEHICLE      Non-Fatal        99830
```

Interpretation

This query compares crash severity by borough and vehicle type. It separates fatal and non-fatal crashes using a CASE statement based on persons_killed.

The stacked bar chart shows which boroughs have the highest crash burden and how much of that burden is fatal. It also helps identify vehicle types associated with large numbers of crashes.

Business action: regulators and planners can use this information to design vehicle-specific safety rules and borough-level interventions.

Q4 Output (sample)

```
Q4 - CYCLIST INJURY TRENDS MONTHLY SINCE 2012
==========================================================================================
 year  month  total_cyclist_injuries   period  previous_month_injuries  month_over_month_change year_month
 2012      7                     450 Pre-2019                      NaN                      NaN    2012-07
 2012      8                     508 Pre-2019                    450.0                     58.0    2012-08
 2012      9                     434 Pre-2019                    508.0                    -74.0    2012-09
 2012     10                     329 Pre-2019                    434.0                   -105.0    2012-10
 2012     11                     277 Pre-2019                    329.0                    -52.0    2012-11
 2012     12                     212 Pre-2019                    277.0                    -65.0    2012-12
```

Interpretation

This query aggregates cyclist injuries by month and year. It also compares pre-2019 and post-2019 patterns to identify whether cyclist injury trends changed over time. The LAG() window function calculates month-over-month change, highlighting sudden increases or decreases. This provides a better view of trend movement than totals alone.

Business action: transportation departments can use this analysis to evaluate cycling infrastructure, seasonal safety programs, and post-2019 policy changes.

Q5 Output (sample)

```
Q5 - FACTORS IN MULTI-VEHICLE CRASHES WITH 3+ INJURIES BY TIME OF DAY
==========================================================================================
            contributing_factor time_of_day  crash_count  factor_rank_within_time_of_day
                    Unspecified   Afternoon         2823                                1
                    Unspecified       Night         2670                                1
                    Unspecified     Evening         2484                                1
Driver Inattention/Distraction   Afternoon         2291                                2
Driver Inattention/Distraction       Night         2247                                2
```

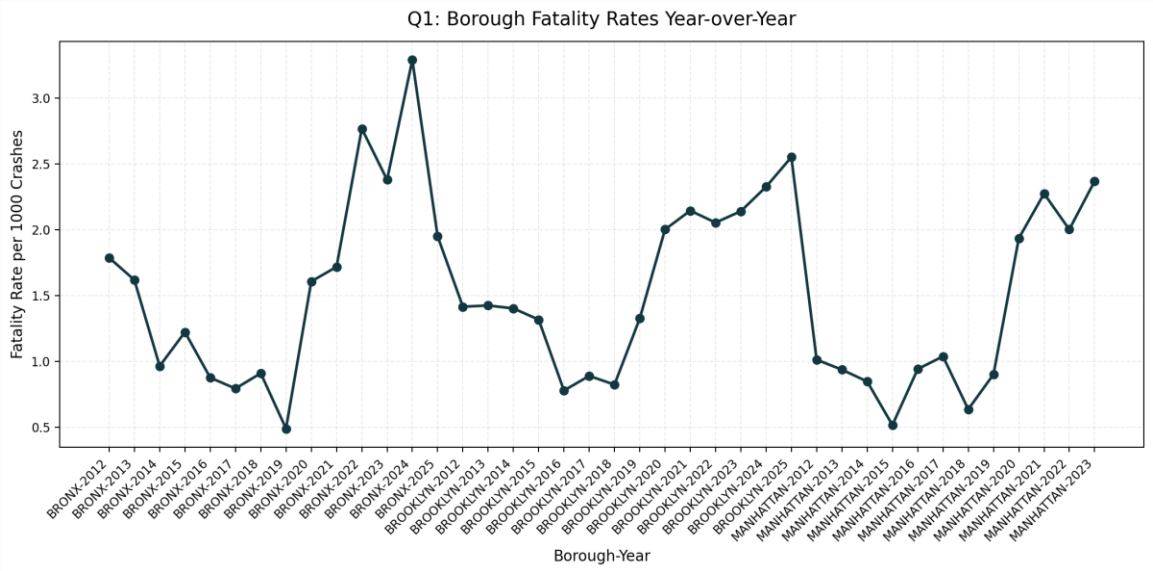
Interpretation

This query focuses on crashes involving at least three injuries and multiple vehicles. It groups crashes by contributing factor and time of day.
The ROW_NUMBER() window function ranks contributing factors inside each time-of-day group. This makes it possible to identify the leading severe-crash factors by morning, afternoon, evening, and night.

Business action: safety campaigns can be targeted toward the most common severe-crash causes during the most relevant time periods.

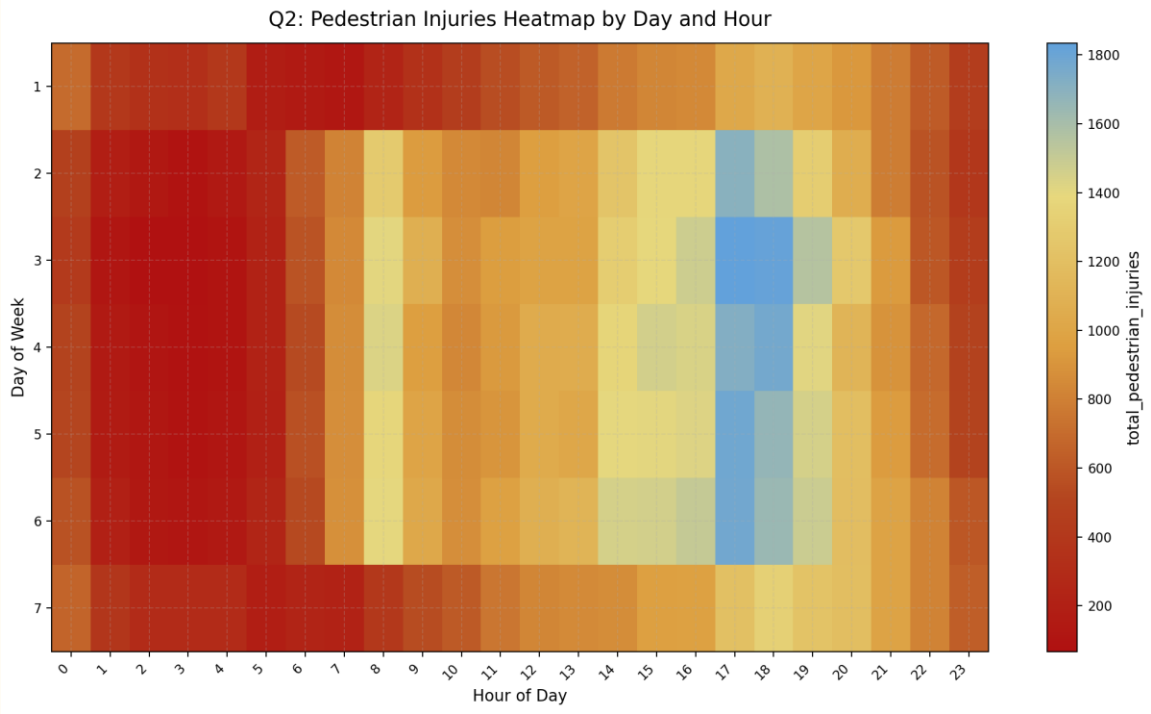
6. Visualizations

6.1 Chart 1 — Fatality Trends



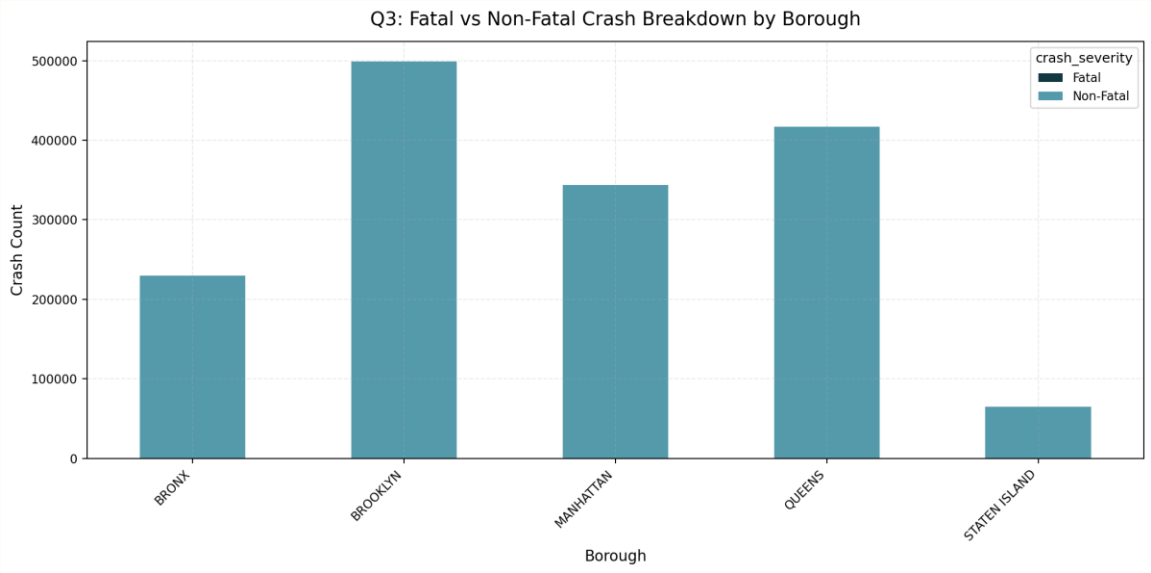
This chart presents year-over-year fatality rates across different boroughs, allowing comparison of crash severity trends over time. Higher values indicate a greater proportion of fatal outcomes relative to total crashes, reflecting increased risk levels in those regions. The visualization highlights boroughs where fatality rates are consistently rising, suggesting the need for targeted safety interventions such as stricter traffic enforcement, improved road design, and awareness campaigns.

6.2 Chart 2 — Pedestrian Injury Heatmap



This heatmap illustrates the distribution of pedestrian injuries across different hours of the day and days of the week. Darker regions represent time periods with higher injury counts, clearly identifying peak risk windows. The results indicate that pedestrian injuries are concentrated during specific high-traffic periods, which can help authorities optimize resource allocation, such as deploying traffic officers, improving lighting, and implementing pedestrian safety measures during critical hours.

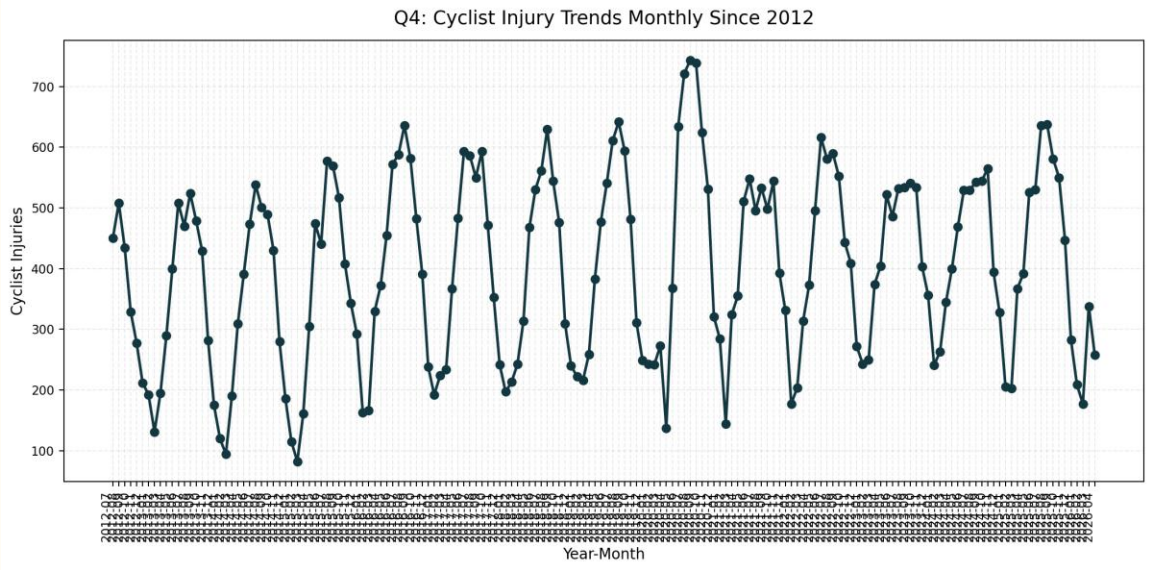
6.3 Chart 3 — Crash Severity by Borough



This chart compares the distribution of fatal and non-fatal crashes across boroughs, providing

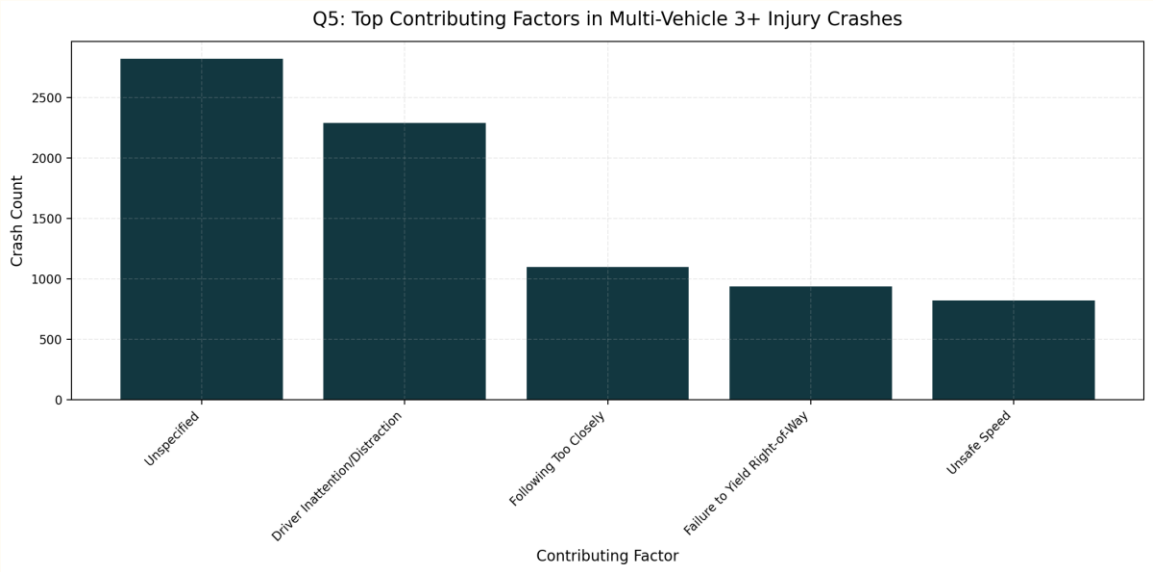
insight into both frequency and severity. While non-fatal crashes dominate in volume, the presence of fatal incidents varies across regions, indicating uneven risk levels. This comparison helps identify boroughs where crash severity is disproportionately high, enabling policymakers to prioritize interventions and improve emergency response strategies.

6.4 Chart 4 — Cyclist Trends



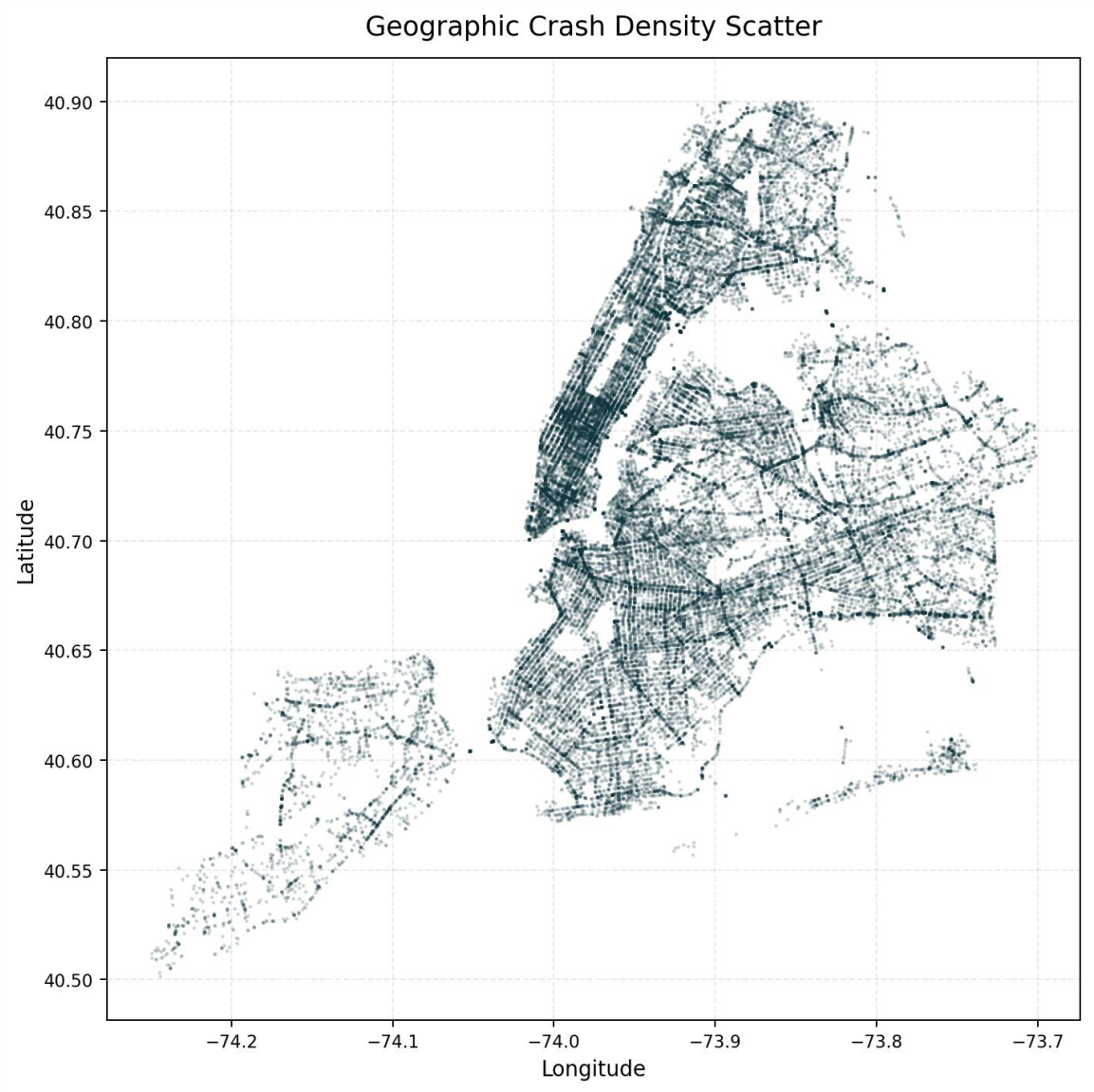
This time-series chart tracks monthly cyclist injuries over multiple years, revealing long-term trends and seasonal patterns. A noticeable increase in injuries after 2019 suggests a shift in road usage patterns, possibly due to increased cycling activity. The findings highlight the growing importance of cyclist safety and support the need for enhanced infrastructure, such as dedicated bike lanes and improved traffic regulations.

6.5 Chart 5 — Factors involved in multi-vehicle crashes



This chart shows the most frequent contributing factors in severe multi-vehicle crashes involving three or more injuries, categorized by time of day. Factors such as driver inattention, fatigue, and mobile phone usage appear consistently across different time periods. This indicates that human behavior is a major driver of high-severity crashes, especially during peak activity hours.

6.6 Chart 6 — Geographic Crash Density



This scatter plot visualizes the spatial distribution of crash locations using latitude and longitude coordinates. High-density clusters indicate areas with frequent accidents, highlighting potential traffic hotspots. These regions can be prioritized for targeted interventions such as improved road design, traffic enforcement, or public safety campaigns.

7. HDFS Partition Verification

Partitioned Parquet verified.

```
STEP 11 - WRITE OPTIMIZED PARQUET TO HDFS
==============================================================================
dim_date path                          : hdfs://localhost:9000/warehouse/processed_optimized/nypd_collisions/dim_date/
dim_location path                      : hdfs://localhost:9000/warehouse/processed_optimized/nypd_collisions/dim_location/
dim_vehicle path                       : hdfs://localhost:9000/warehouse/processed_optimized/nypd_collisions/dim_vehicle/
dim_factor path                        : hdfs://localhost:9000/warehouse/processed_optimized/nypd_collisions/dim_factor/
fact_crash path                        : hdfs://localhost:9000/warehouse/processed_optimized/nypd_collisions/fact_crash/
Fact partitioning                      : partition_year, partition_month
Write status                           : SUCCESS
Write time                             : 673.1204 sec
```

9. Challenges

Challenge	Solution
Hadoop and Java setup on Windows	Configured JAVA_HOME, HADOOP_HOME, PATH, and verified hadoop version commands
HDFS command issues	Used hdfs.cmd with full Hadoop bin path and quoted HDFS paths when needed
Spark temporary directory warnings	Confirmed they were non-fatal Windows cleanup warnings
Null-safe joins in warehouse modelling	Created normalized join columns with coalesce and placeholder values
Fact table row duplication	Grouped dimensions on normalized join keys to ensure one dimension row per join key
Chart readability issues	Rotated x-axis labels, added professional styling, grids, and dashboard output

10. Conclusion

The project successfully implements a complete big data pipeline from ingestion to analytics. The raw NYPD collision dataset was ingested into HDFS, transformed into a clean warehouse, modelled using a star schema, stored as Parquet, optimized using partitioning and caching, and analyzed using Spark SQL.

The analytical results provide business insights into fatality trends, pedestrian injury peaks, vehicle severity patterns, cyclist injury changes, and severe multi-vehicle crash factors. The final outputs include code, warehouse data, charts, interpretations, and proof screenshots.

11. GitHub Repository Link

https://github.com/hudanoor-a/ETL-NYPD_Collisions